Architecture technique – Plateforme de monitoring Smart Building

1. Choix techniques

1.1 Vue d'ensemble de la stack

Composant	Technologie	Version	Rôle dans la plateforme
Moteur de recherche et d'indexation	Elasticsearch	8.x	Stocke et indexe les logs des capteurs, exécute les recherches et les agrégations
Pipeline d'ingestion	Logstash	8.x	Lit les fichiers déposés, les parse (CSV, JSON), les enrichit et les envoie à Elasticsearch
Visualisation	Kibana	8.x	Dashboards (température, alertes, énergie) intégrés dans l'application
Application web	Flask (Python)	3.x	Interface utilisateur, API REST, logique métier
Base de données applicative	MongoDB	7.x	Métadonnées des fichiers, utilisateurs, historique des recherches, règles et alertes
Cache et messagerie	Redis	7.x	Cache des requêtes, sessions, messagerie Pub/Sub avec le worker d'alertes
Conteneurisation	Docker et Docker Compose	–	Déploiement de tous les services en une commande

1.2 Pourquoi trois bases de données ?

Chaque base répond à un besoin différent :

- **Elasticsearch** est optimisé pour la recherche plein texte et les agrégations sur de gros volumes de données horodatées. Il est adapté aux logs, mais pas aux données qui changent souvent (un utilisateur, le statut d'une alerte).
- **MongoDB** stocke les données de l'application, peu volumineuses mais modifiées régulièrement : comptes, règles d'alerte, statut des fichiers importés. Son modèle orienté documents (JSON) évite un schéma rigide.
- **Redis** garde en mémoire des données temporaires auxquelles on accède très souvent : sessions, résultats de recherche récents, messages entre services. Il répond en moins d'une milliseconde.

1.3 Flask plutôt que Django

Critère	Flask	Django
Taille et complexité	Léger, on ajoute uniquement ce dont on a besoin	Complet, mais beaucoup de fonctionnalités inutiles ici (ORM SQL, admin)
Base de données	Aucune imposée, s'intègre simplement avec MongoDB	ORM pensé pour les bases SQL
Temps réel (WebSocket)	Flask-SocketIO, simple à mettre en place	Django Channels, plus complexe
Prise en main	Rapide pour un projet de quelques semaines	Plus longue

Choix : Flask, car le projet n'utilise pas de base SQL et a besoin d'une API simple et de WebSockets.

1.4 Un worker d'alertes séparé

La vérification des règles d'alerte s'exécute dans un **conteneur séparé** (worker Python), et non dans l'application Flask :

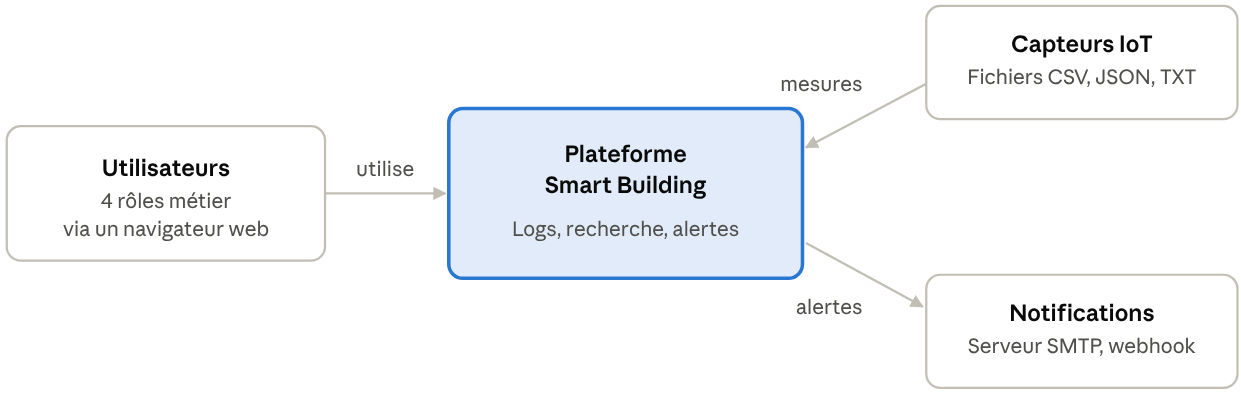
- l'application web reste rapide : elle ne bloque pas pendant les vérifications ;
- le worker tourne en continu, même quand personne n'utilise l'interface ;
- les deux communiquent via Redis (publication des nouvelles alertes), ce qui les rend indépendants.

2. Architecture globale

L'architecture est décrite selon le modèle **C4**, à deux niveaux de détail.

2.1 Niveau 1 : diagramme de contexte

La plateforme reçoit les mesures des capteurs et notifie les équipes



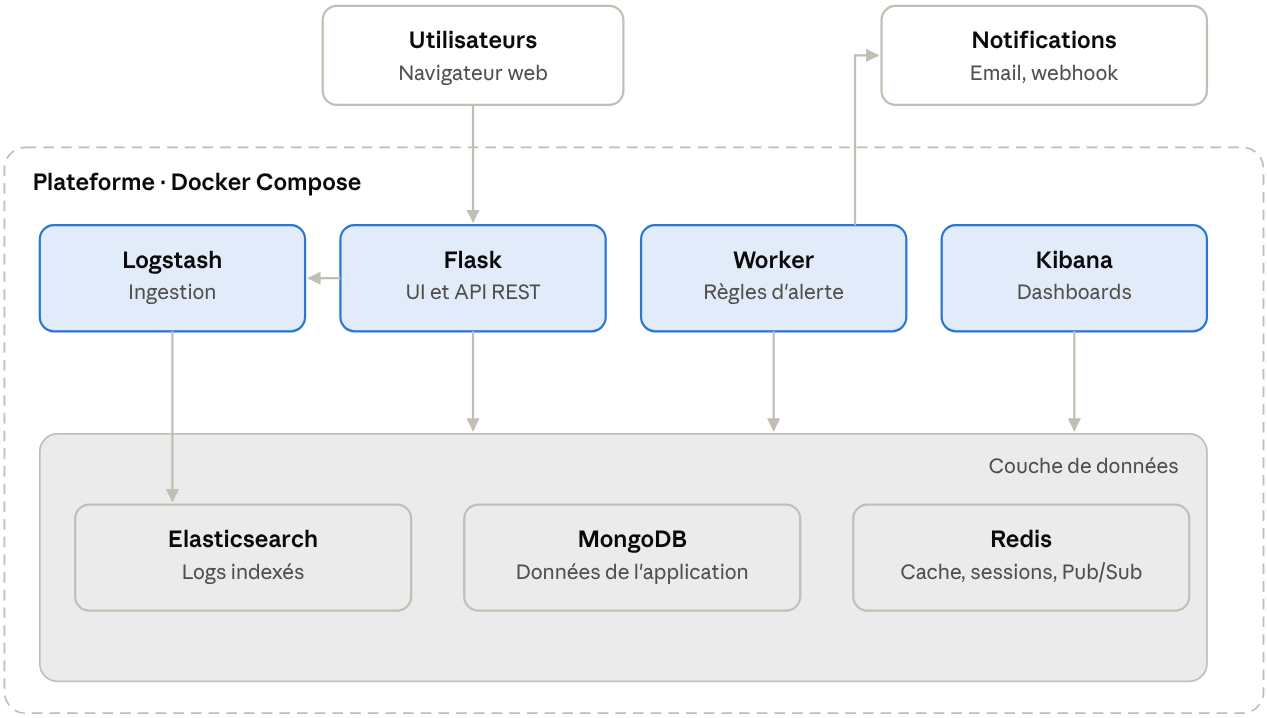
C4 niveau 1 · diagramme de contexte

La plateforme interagit avec trois éléments extérieurs :

- les **utilisateurs** (responsable du bâtiment, technicien, gestionnaire énergétique, administrateur), via un navigateur web ;
- les **capteurs IoT**, dont les mesures arrivent sous forme de fichiers de logs (CSV, JSON, TXT) ;
- le **service de notification** (serveur SMTP, webhook), qui reçoit les alertes envoyées par la plateforme.

2.2 Niveau 2 : diagramme de conteneurs

Quatre services s'appuient sur trois bases de données



C4 niveau 2 · diagramme de conteneurs

La plateforme est composée de sept conteneurs Docker, organisés en deux couches : les **services** (Logstash, Flask, worker d'alertes, Kibana) et les **données** (Elasticsearch, MongoDB, Redis).

Conteneur	Rôle	Communique avec
Flask	Interface web, API REST, validation des fichiers	Elasticsearch (recherche, statistiques), MongoDB (fichiers, utilisateurs, historique), Redis (cache, sessions), dossier partagé avec Logstash
Logstash	Lit les fichiers déposés, les parse et les envoie à Elasticsearch	Dossier partagé (lecture), Elasticsearch (écriture)

Conteneur	Rôle	Communique avec
Worker d'alertes	Vérifie les règles d'alerte toutes les minutes et déclenche les notifications	Elasticsearch (lecture des mesures), MongoDB (règles, alertes), Redis (publication des nouvelles alertes), service de notification
Kibana	Dashboards de visualisation, intégrés dans l'application	Elasticsearch (lecture)
Elasticsearch	Stockage et indexation des logs	–
MongoDB	Données de l'application	–
Redis	Cache, sessions, messagerie Pub/Sub	–

Flux principal : l'utilisateur importe un fichier via Flask → Flask le valide et le dépose dans un dossier partagé → Logstash le lit et l'indexe dans Elasticsearch → les données sont consultables par la recherche, le tableau de bord et Kibana, et analysées par le worker d'alertes.

3. Déploiement Docker

L'ensemble de la plateforme est déployé avec **Docker Compose** : une seule commande (`docker compose up -d`) démarre les sept conteneurs.

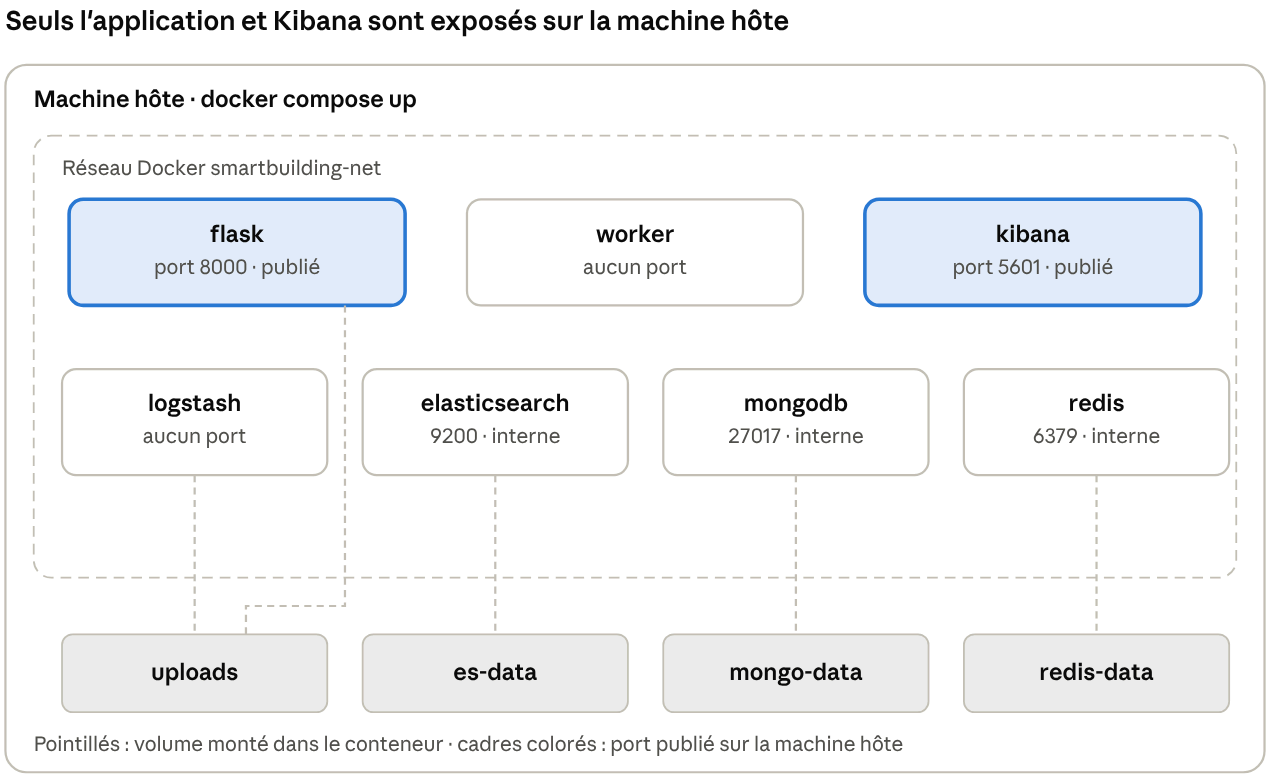


Diagramme de déploiement Docker

3.1 Conteneurs

Service	Image	Port	Volume	Dépend de
`elasticsearch`	elasticsearch 8.x (officielle)	9200 (interne)	`es-data`	–
`logstash`	logstash 8.x (officielle)	–	`uploads` (lecture), pipelines	elasticsearch
`kibana`	kibana 8.x (officielle)	5601 (publié)	–	elasticsearch
`mongodb`	mongo 7	27017 (interne)	`mongo-data`	–
`redis`	redis 7	6379 (interne)	`redis-data`	–

Service	Image	Port	Volume	Dépend de
`flask`	image construite (Dockerfile Python)	8000 (publié)	`uploads` (écriture)	elasticsearch, mongodb, redis
`worker`	même image que Flask, autre commande	–	–	elasticsearch, mongodb, redis

3.2 Réseau

Tous les conteneurs sont connectés au réseau `smartbuilding-net` et se joignent par leur nom de service. Seuls l'application (8000) et Kibana (5601) sont accessibles depuis la machine hôte : les bases de données ne sont pas exposées.

3.3 Volumes

- `es-data`, `mongo-data`, `redis-data` : conservent les données quand les conteneurs redémarrent ;
- `uploads` : dossier partagé entre Flask (qui y dépose les fichiers validés) et Logstash (qui le surveille).

3.4 Ordre de démarrage et health checks

Chaque base de données possède un **health check** : Docker vérifie régulièrement qu'elle répond (`_cluster/health` pour Elasticsearch, `ping` pour MongoDB et Redis). Les services qui en dépendent utilisent `depends_on` avec la condition `service_healthy` : Flask ne démarre que lorsque Elasticsearch, MongoDB et Redis sont prêts. Flask expose lui-même un endpoint `/health`.

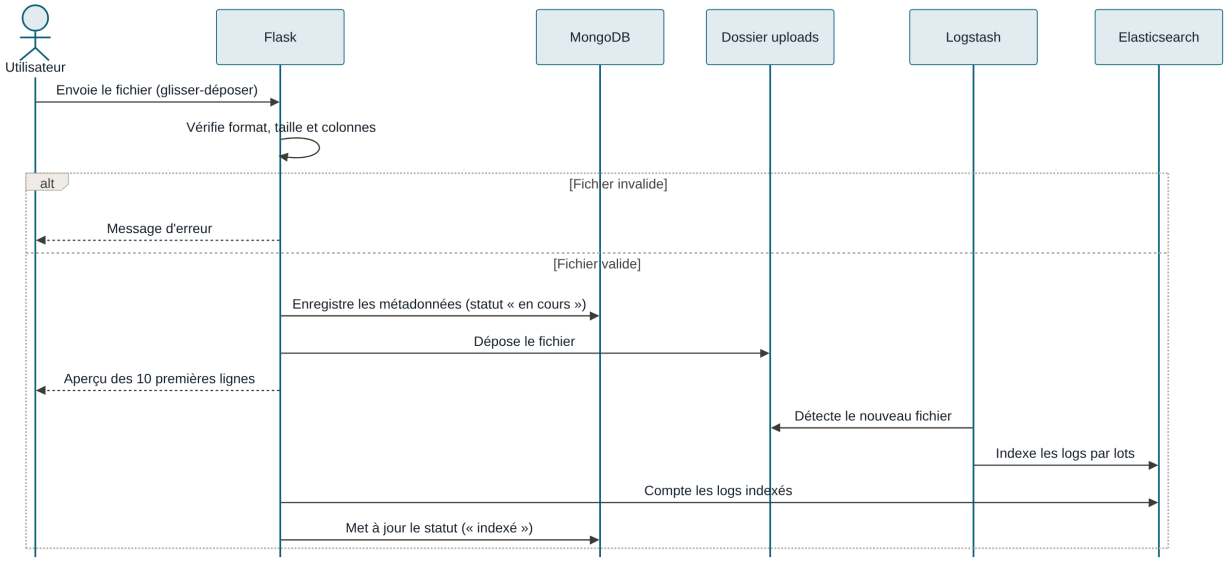
3.5 Configuration

Les paramètres (mots de passe, adresses des services, mémoire allouée à Elasticsearch) sont définis dans un fichier `.env`, non versionné. Un fichier `.env.example` documente les variables attendues.

4. Diagrammes de séquence

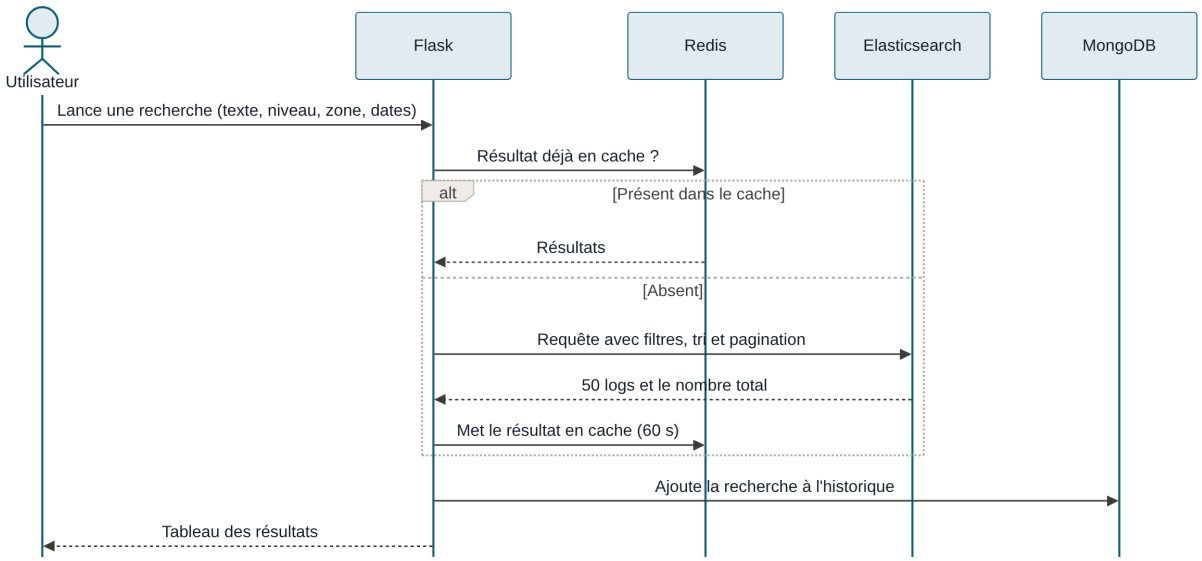
Les trois scénarios principaux de la plateforme sont détaillés ci-dessous. Chaque colonne est un composant, le temps s'écoule de haut en bas ; les flèches pleines sont des requêtes, les flèches en pointillés des réponses.

4.1 Import d'un fichier de logs



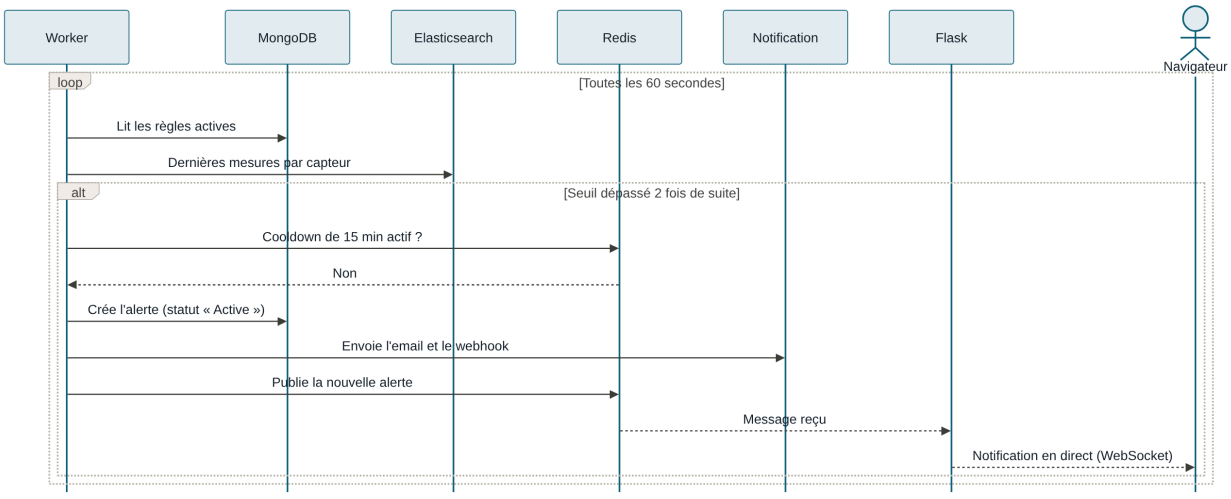
Flask valide le fichier avant toute ingestion : format (CSV, JSON, TXT), taille (100 Mo maximum) et présence des colonnes obligatoires (RM-15, RM-16). Un fichier valide est enregistré dans MongoDB puis déposé dans le dossier partagé. L'indexation est ensuite réalisée **de façon asynchrone** par Logstash : l'utilisateur n'attend pas la fin de l'ingestion pour continuer à utiliser l'application. Flask suit la progression en comptant les logs indexés dans Elasticsearch, puis met à jour le statut du fichier.

4.2 Recherche de logs



Avant d'interroger Elasticsearch, Flask vérifie si la même recherche a été faite récemment : si oui, le résultat est servi depuis le **cache Redis**, ce qui évite une requête coûteuse. Sinon, Elasticsearch renvoie la page de 50 résultats et le nombre total, et le résultat est mis en cache pendant 60 secondes. Chaque recherche est ajoutée à l'historique de l'utilisateur dans MongoDB (RM-20).

4.3 Déclenchement d'une alerte



Le worker vérifie les règles actives toutes les 60 secondes, au rythme d'envoi des capteurs (RM-04). Une alerte n'est créée que si le seuil est dépassé sur **deux mesures consécutives** (RM-11) et qu'aucune notification n'a été envoyée pour ce capteur dans les 15 dernières minutes (RM-12). Le cooldown est stocké dans Redis sous forme de clé à durée de vie limitée. Le worker publie ensuite l'alerte dans Redis ; Flask la reçoit et la transmet en direct au navigateur par WebSocket.

5. Modèles de données

5.1 Elasticsearch : mapping des logs

Les logs sont stockés dans des index mensuels (`sensor-logs-2026.10` , `sensor-logs-2026.11` …) créés automatiquement à partir d'un **index template**. Une politique ILM supprime les index de plus de 90 jours (RM-19).

```
{
  "index_patterns": ["sensor-logs-*"],
  "template": {
    "mappings": {
      "properties": {
        "@timestamp":  { "type": "date" },
        "sensor_id":   { "type": "keyword" },
        "zone":        { "type": "keyword" },
        "floor":       { "type": "integer" },
        "room":        { "type": "keyword" },
        "type":        { "type": "keyword" },
        "value":       { "type": "float" },
        "unit":        { "type": "keyword" },
        "level":       { "type": "keyword" },
        "message":     { "type": "text", "fields": { "raw": { "type":
"keyword" } } },
        "source_file": { "type": "keyword" }
      }
    }
  }
}
```

- Les champs `keyword` (capteur, zone, niveau) servent aux **filtres exacts** et aux agrégations (comptage par zone, heatmap).
- Le champ `message` est de type `text` pour la **recherche plein texte**, avec un sous-champ `keyword` pour le tri.
- `value` est numérique pour calculer moyennes, maximums et seuils.

5.2 MongoDB : collections

Collection	Contenu
`users`	Comptes, rôles, mot de passe chiffré
`files`	Métadonnées des fichiers importés
`search_history`	Historique des recherches de chaque utilisateur
`alert_rules`	Règles d'alerte configurées
`alerts`	Alertes déclenchées et leur cycle de vie

Exemple de document `files` :

```
{
  "_id": "6719f2c4a1b2c3d4e5f60718",
  "filename": "capteurs_2026-10-23.csv",
  "format": "csv",
  "size_bytes": 44669337,
  "uploaded_by": "6719e0aa1b2c3d4e5f607182",
  "uploaded_at": "2026-10-24T21:58:00Z",
  "status": "indexed",
  "lines_total": 168920,
  "lines_invalid": 714,
  "logs_indexed": 168206
}
```

Exemple de document `alert_rules` :

```
{
  "_id": "6719f8d2a1b2c3d4e5f60801",
  "name": "Surchauffe salle",
  "sensor_type": "temperature",
  "operator": ">",
  "threshold": 30,
  "consecutive": 2,
  "severity": "CRITICAL",
  "channels": ["email", "webhook"],
  "cooldown_minutes": 15,
  "enabled": true
}
```

Exemple de document `alerts` :

```
{
  "_id": "671a0c11a1b2c3d4e5f60902",
  "rule_id": "6719f8d2a1b2c3d4e5f60801",
  "sensor_id": "TEMP-E2-S204",
  "value": 30.8,
  "status": "acknowledged",
  "triggered_at": "2026-10-23T15:02:00Z",
  "acknowledged_by": "6719e0aa1b2c3d4e5f607183",
  "acknowledged_at": "2026-10-23T15:09:00Z",
  "resolved_at": null
}
```

5.3 Redis : structure des clés

Clé	Type Redis	Durée de vie	Usage
`cache:search:<empreinte>`	String (JSON)	60 s	Résultat d'une recherche récente
`session:<id>`	Hash	8 h	Session de l'utilisateur connecté
`cooldown:<règle>:<capteur>`	String	15 min	Empêche les notifications en double (RM-12)
`ratelimit:<utilisateur>`	Compteur	60 s	Limite le nombre de requêtes à l'API par minute
`alerts`	Canal Pub/Sub	–	Transmet les nouvelles alertes du worker vers Flask

Toutes les clés ont une **durée de vie (TTL)** : Redis ne contient que des données temporaires. Une perte du cache n'entraîne aucune perte de données.